Milestone 2 (Draft)

CSC 676 - Spring 2026

Team Alias: BayConnect

Team Members:

Nate Mugambi
nmugambi@sfsu.edu
GitHub: natemugambi
Tia Parker
tparker1@sfsu.edu
GitHub: tzzzp
Sean Wang
swang29@sfsu.edu
GitHub: sw8ng
Devin Yuan
dyuan3@sfsu.edu
GitHub: BinaryFallow

Repository: Link

Version History:

Version	Description	Date
v0.1	Initial draft for document describing normalization decisions, and more attributes added into database schema diagram	2026-03-10
v0.2	Second draft of Milestone 2. Changes from the first draft are based on instructor feedback.	2026-04-04

Table Of Contents

1. Cover Page

2. Table Of Contents

3. Introduction

4. Transition from Conceptual Model to Relations

5. Normalization Process

5. Higher Normal Forms

7. Finalized EER Diagram

Introduction

- **Project Overview:** The Transit Incident & Maintenance Reporting System improves how passengers experience and interact with local rail platforms like BART and Muni.
- **System Purpose:** The primary goal is to provide a platform where users can report hazards such as equipment failure and safety concerns. Which are then categorized and dispatched to the appropriate personnel for the problem to be resolved.
- **Milestone 2 Objectives:** This document aims to transition the conceptual EER model into a relational schema. The normalization process is used to eliminate redundancy in data.
- **Design Scope:** The relational schema will cover user authentication for multiple roles, station-specific incident tracking, and the relationships between transit agencies and their physical infrastructure.

Transition from Conceptual Model to Relations

Entities identified in the conceptual model will be mapped to a relational schema:

- **Mapping Entity Types:** strong entities from the conceptual design will be mapped to a relation
- **Specialization/Generalization:** implement a “multiple relation” map for User hierarchy
    - Authentication data is centralized in the User table
    - Role-specific attributes are stored in Admin, Employee, and Officer tables, linked via Foreign Key to user_id
- **Relationship Mapping:** Binary 1:N relationships are represented by Foreign Keys
- **Multi-valued Attributes:** Can have multiple values (i.e., hazards associated with an incident moved to a separate table)

Normalization Process

The goal of the Normalization process is to ensure the schema is free from abnormalities. Normalization organizes the data into more structured tables so that the connection between the different aspects of the database is clearer to an observer. It helps keep the data consistent and reliable across the database.

Before the Normalization process, the first thing the team did was to create a schema of our work in Milestone 1. We made sure to improve our work in the first milestone based on feedback before we start fine-tuning the schema by including all the attributes and correcting cardinality.

2NF can only be done after achieving 1NF, 3NF can only be done after achieving 2NF and so on and so forth. Before each Normalization stage, we scan through our conceptual model to see which chunks can be broken down into smaller more manageable pieces.

Normalization Analysis Table

Link to table

Table	Candidate Keys	Prime Attributes	Non-prime Attributes	Functional Dependencies	2NF (partial dependencies?)	3NF (transitive?)	BCNF	4NF
User	user_id, username, email	user_id, username, email	password, first_name, last_name	user_id → username, email, password, first_name, last_name; username → user_id, email, password, first_name, last_name; email → user_id, username, password, first_name, last_name	No partial dependencies (single-column candidate keys)	Each FD left side is a candidate key (superkey). No transitive dependencies.	Yes	Yes
Admin	admin_id, user_id	admin_id, user_id	role_name, access_level, admin_status	admin_id → role_name, access_level, admin_status; user_id → role_name, access_level,	Single-column candidate keys → automatically 2NF	Left sides are candidate keys → OK	Yes	Yes

				admin_status				
Employee	employee_id, user_id	employee_id, user_id	station_id, job_title, employment_status, hire_date	employee_id → station_id, job_title, employment_status, hire_date; user_id → station_id, job_title, employment_status, hire_date	Single-column candidate keys → automatically 2NF	Left sides are candidate keys → OK	Yes	Yes
Officer	officer_id, user_id, badge_number	officer_id, user_id, badge_number	rank, assigned_shift	officer_id → rank, assigned_shift; user_id → rank, assigned_shift; badge_number → rank, assigned_shift	Single-column candidate keys → automatically 2NF	Left sides are candidate keys → OK	Yes	Yes
Agency	agency_id, name	agency_id, name	contact_phone, hq_street, hq_city, hq_state, hq_zip	agency_id → contact_phone, hq_street, hq_city, hq_state, hq_zip; name → contact_phone, hq_street, hq_city, hq_state, hq_zip	Single-column candidate keys → automatically 2NF	Left sides are candidate keys → OK	Yes	Yes
Station	station_id	station_id	name, city, zone_cod	station_id → name, city,	Single-column PK →	Left side is superkey	Yes	Yes

			e	zone_code	automatically 2NF	→ OK		
Mode	mode_id, name	mode_id, name	description, typical_use_case	mode_id → description, typical_use_case; name → description, typical_use_case	Single-column candidate keys → automatically 2NF	Left sides are candidate keys → OK	Yes	Yes
Report_Category	category_id, name	category_id, name	description, escalation_flag	category_id → description, escalation_flag; name → description, escalation_flag	Single-column candidate keys → automatically 2NF	Left sides are candidate keys → OK	Yes	Yes
Incident_Status	status_id, name	status_id, name	description, display_order	status_id → description, display_order; name → description, display_order	Single-column candidate keys → automatically 2NF	Left sides are candidate keys → OK	Yes	Yes
Vehicle_Status	status_id, name	status_id, name	description, operational_flag	status_id → description, operational_flag; name → description, operational_flag	Single-column candidate keys → automatically 2NF	Left sides are candidate keys → OK	Yes	Yes

Hazards	hazards_id, name	hazards_id, name	description, severity_class	hazards_id → description, severity_class; name → description, severity_class	Single-column candidate keys → automatically 2NF	Left sides are candidate keys → OK	Yes	Yes
Priority_Level	priority_id	priority_id	name, description, severity_rank	priority_id → name, description, severity_rank	Single-column PK → automatically 2NF	Left side is superkey → OK	Yes	Yes
Responder	responder_id, name	responder_id, name	description, responder_category	responder_id → description, responder_category; name → description, responder_category	Single-column candidate keys → automatically 2NF	Left sides are candidate keys → OK	Yes	Yes
Data_Feed	feed_Id, feed_name	feed_Id, feed_name	feed_type	feed_Id → feed_type; feed_name → feed_type	Single-column candidate keys → automatically 2NF	Left sides are candidate keys → OK	Yes	Yes
Line	line_id, name	line_id, name	agency_id, mode_id, line_color, service_pattern	line_id → agency_id, mode_id, line_color, service_pattern; name → agency_id, mode_id, line_color, service_pattern	Single-column candidate keys → automatically 2NF	Left sides are candidate keys → OK	Yes	Yes

Weather	weather_id	weather_id	feed_id, status, temperature, recorded_timestamp	weather_id → feed_id, status, temperature, recorded_timestamp	Single-column PK → automatically 2NF	Left side is superkey → OK	Yes	Yes
Incident_Report	incident_id	incident_id	user_id, status_id, category_id, weather_id, priority_id, description, reported_timestamp, incident_source	incident_id → user_id, status_id, category_id, weather_id, priority_id, description, reported_timestamp, incident_source	Single-column PK → automatically 2NF	Left side is superkey; no transitive dependencies because non-prime FKs do not determine other non-prime attributes	Yes	Yes
Incident_Station	(incident_id, station_id)	incident_id, station_id	location_note, area_type, linked_timestamp	(incident_id, station_id) → location_note, area_type, linked_timestamp	No partial dependency: both parts needed to determine non-prime	Left side is superkey (the only candidate key) → OK	Yes	Yes
Incident_Train	(incident_id, train_id)	incident_id, train_id	train_section_note, impact_type, linked_timestamp	(incident_id, train_id) → train_section_note, impact_type, linked_timestamp	No partial dependency	Left side is superkey → OK	Yes	Yes
Incident_Photo	(incident_id, photo_sequence)	incident_id, photo_sequence	timestamp, photo_url, caption	(incident_id, photo_sequence) → timestamp	No partial dependency: photo_sequence	Left side is superkey → OK	Yes	Yes

				, photo_url, caption	needed to identify a specific photo			
Incident_ History	history_id	history_id	incident_id, timestamp, action_desc, action_type	history_id → incident_id, timestamp, action_desc, action_type	Single-column PK → automatically 2NF	Left side is superkey → OK	Yes	Yes
Incident_ Hazard	(incident_id, hazard_id)	incident_id, hazard_id	hazard_severity, Incident_ Hazardcol, mitigation_status	(incident_id, hazard_id) → hazard_severity, Incident_ Hazardcol, mitigation_status	No partial dependency	Left side is superkey → OK	Yes	Yes
Car	car_id, car_name	car_id, car_name	agency_id, status_id, capacity, car_type	car_id → agency_id, status_id, capacity, car_type; car_name → agency_id, status_id, capacity, car_type	Single-column candidate keys → automatically 2NF	Left sides are candidate keys → OK	Yes	Yes
Train	train_id, train_code	train_id, train_code	agency_id, status_id, capacity, service_type	train_id → agency_id, status_id, capacity, service_type; train_code	Single-column candidate keys → automatically 2NF	Left sides are candidate keys → OK	Yes	Yes

				→ agency_id, status_id, capacity, service_type				
Train_Car	(train_id, car_id)	train_id, car_id	car_order, coupled_timestamp, car_role	(train_id, car_id) → car_order, coupled_timestamp, car_role	No partial dependency	Left side is superkey → OK	Yes	Yes
Trip	trip_id	trip_id	train_id, line_id, scheduled_start, scheduled_end, trip_status	trip_id → train_id, line_id, scheduled_start, scheduled_end, trip_status	Single-column PK → automatically 2NF	Left side is superkey → OK	Yes	Yes
Stop	stop_id, (trip_id, station_id), (trip_id, sequence_number)	stop_id, trip_id, station_id, sequence_number	scheduled_arrival, scheduled_departure	stop_id → scheduled_arrival, scheduled_departure; (trip_id, station_id) → scheduled_arrival, scheduled_departure; (trip_id, sequence_number) → scheduled_arrival, scheduled_departure	No partial dependency (for composite keys, both parts needed)	For each FD, left side is a candidate key (superkey) → OK	Yes	Yes
Live_Feed	feed_id	feed_id	stop_id, update_timestamp, estimated	feed_id → stop_id, update_timestamp,	Single-column PK → automatic	Left side is superkey → OK	Yes	Yes

			_arrival_time, capacity_percentage	estimated_arrival_time, capacity_percentage	ally 2NF			
Delay_Propagation	propogation_id	propogation_id	incident_id, trip_id, stop_id, delay_minutes, calculated_timestamp, impact_scope	propogation_id → incident_id, trip_id, stop_id, delay_minutes, calculated_timestamp, impact_scope	Single-column PK → automatically 2NF	Left side is superkey → OK	Yes	Yes
Station_Agency	(station_id, agency_id)	station_id, agency_id	service_start_date, service_end_date, service_notes	(station_id, agency_id) → service_start_date, service_end_date, service_notes	No partial dependency	Left side is superkey → OK	Yes	Yes
Line_Station	(line_id, station_id), (line_id, sequence_number)	line_id, station_id, sequence_number	platform_info, destination_label	(line_id, station_id) → sequence_number, platform_info, destination_label; (line_id, sequence_number) → station_id, platform_info, destination_label	No partial dependency (both composite keys are minimal)	Left sides are superkeys (candidate keys) → OK	Yes	Yes

Saved_Station	saved_id, (user_id, station_id)	saved_id, user_id, station_id	created_timestamp, custom_label, notification_enabled	saved_id → created_timestamp, custom_label, notification_enabled; (user_id, station_id) → created_timestamp, custom_label, notification_enabled	Single-column candidate key (saved_id) → automatically 2NF; composite key also has no partial dependency	Left sides are superkeys → OK	Yes	Yes
Upvote	upvote_id, (user_id, incident_id)	upvote_id, user_id, incident_id	created_timestamp, vote_source, active_flag	upvote_id → created_timestamp, vote_source, active_flag; (user_id, incident_id) → created_timestamp, vote_source, active_flag	Single-column candidate key → automatically 2NF; composite key has no partial dependency	Left sides are superkeys → OK	Yes	Yes
Notification_Preference	preference_id, (user_id, notification_type)	preference_id, user_id, notification_type	enabled_flag, delivery_channel	preference_id → enabled_flag, delivery_channel; (user_id, notification_type) → enabled_flag, delivery_channel	Single-column candidate key → automatically 2NF; composite key has no partial dependency	Left sides are superkeys → OK	Yes	Yes

Notification	notification_id	notification_id	user_id, incident_id, line_id, created_timestamp, delivery_status, notification_type	notification_id → user_id, incident_id, line_id, created_timestamp, delivery_status, notification_type	Single-column PK → automatically 2NF	Left side is superkey; no transitive dependencies (incident_id and line_id are FKs, they do not determine other non-prime)	Yes	Yes
Dispatch_Low_Level	low_level_dispatch_id	low_level_dispatch_id	incident_id, employee_id, officer_id, dispatched_timestamp, dispatch_status, response_notes	low_level_dispatch_id → incident_id, employee_id, officer_id, dispatched_timestamp, dispatch_status, response_notes	Single-column PK → automatically 2NF	Left side is superkey → OK	Yes	Yes
Dispatch_High_Level	high_level_dispatch_id	high_level_dispatch_id	incident_id, responder_id, station_id, dispatched_timestamp, dispatch_status, priority_vote	high_level_dispatch_id → incident_id, responder_id, station_id, dispatched_timestamp, dispatch_status, priority_vote	Single-column PK → automatically 2NF	Left side is superkey → OK	Yes	Yes

Fare_Log	log_id	log_id	incident_id, user_id, station_id, detected_timestamp, fare_amount, detection_method	log_id → incident_id, user_id, station_id, detected_timestamp, fare_amount, detection_method	Single-column PK → automatically 2NF	Left side is superkey → OK	Yes	Yes
Incident_Update	update_id	update_id	incident_id, user_id, comment_text, created_timestamp, visibility_status	update_id → incident_id, user_id, comment_text, created_timestamp, visibility_status	Single-column PK → automatically 2NF	Left side is superkey → OK	Yes	Yes
Vote	vote_id, (user_id, update_id)	vote_id, user_id, update_id	vote_type, created_timestamp, active_flag	vote_id → vote_type, created_timestamp, active_flag; (user_id, update_id) → vote_type, created_timestamp, active_flag	Single-column candidate key → automatically 2NF; composite key has no partial dependency	Left sides are superkeys → OK	Yes	Yes

Normalization Changes

- 1NF:
  - All attributes are atomic. Each table has a primary key.
  - Added a composite primary key (incident, photo sequence) to Incident_Photo as it was weak before. A photo is identified by its incident and sequence number.
  - Split headquarters_location into hq_street, hq_ city, hq_state, and hq_zip to make address components atomic.
  - Added first_name and last_name to User. Full name will be derived and not stored.
- 2NF:
  - We examined every table with a composite candidate key (Station_Agency, Line_station, Incident_Station, Incident_Train, Incident_photo, Incident_Hazard, Train_Car, Stop, Saved_Station, Upvote, Notification_Preference, Vote). FOr each, we listed the functional dependency in the table above. In all cases, every non-prime attribute depends on the whole composite key and not on a subset. Therefore there is no partial dependencies present, and 2NF is satisfied.
- 3NF:
  - For every functional dependency x->y, we verified that either x is a superkey or y is a prime attribute as shown in the table. No functional dependency fails the 3NF condition.
- BCNF:
  - Every determinant (x) is a candidate key. No determinant is a non-superkey. Therefore the schema is in BCNF
- 4NF:
  - There are no multi-valued dependencies. For example, all multivalued attributes such as multiple hazard or photos per incidents are in their own tables (Incident_Hazard, Incident_Photo). The schema in in 4NF.

Finalized EER Diagram

